



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E
INDUSTRIAL



Sistema de Registro de Asistencia en C++ mediante Recorridos de Árboles Binarios

Nombre:

Joselyn Amparo Redroban Chimbo

Carrera:

Software 3 “B”

Asignatura:

Estructura de Datos

Docente:

Ing. José Rubén Caiza

Enero – Junio
2026

1. Resumen

El presente informe técnico describe la implementación de recorridos en árboles binarios utilizando los lenguajes C++ y Java. El trabajo se basa en la construcción de un árbol binario con valores numéricos, al cual se le aplican los recorridos preorden, inorden, postorden y BFS. Además, se implementan métodos para contar la cantidad total de nodos y el número de hojas del árbol.

La práctica permite comprender cómo funcionan las estructuras jerárquicas dentro de la programación. También se relaciona el uso de árboles binarios con un sistema de registro de asistencia, donde los módulos pueden organizarse de manera similar a una estructura de árbol. De esta forma, el informe integra teoría, implementación y análisis de resultados.

2. Objetivo

a. Objetivo general

Implementar y analizar recorridos de árboles binarios en C++ y Java, aplicando sus conceptos a la organización de módulos de un sistema de registro de asistencia.

b. Objetivos específicos

- Construir un árbol binario con nodos enteros y aplicar los recorridos preorden, inorden, postorden y BFS.
- Modificar el árbol inicial agregando nuevos nodos para observar cambios en los recorridos.
- Implementar métodos para contar el número total de nodos y hojas del árbol.
- Comparar la implementación del árbol binario en C++ y Java.
- Relacionar los recorridos de árboles con una aplicación práctica en un sistema web de asistencia.

3. Introducción

Los árboles binarios son estructuras de datos no lineales que permiten representar información de forma jerárquica. A diferencia de estructuras lineales como arreglos, pilas o colas, los árboles organizan sus datos en niveles, partiendo de un nodo raíz y extendiéndose hacia nodos hijos. Esta característica los hace útiles para representar menús, decisiones, expresiones, estructuras de archivos y módulos de sistemas informáticos.

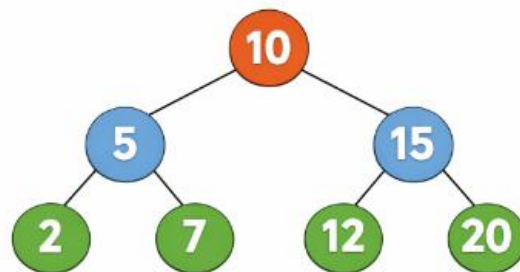
En el estudio de estructuras de datos, los recorridos de árboles son fundamentales porque permiten visitar cada nodo siguiendo un orden determinado. Los recorridos más comunes son preorden, inorden, postorden y BFS. Cada uno tiene una finalidad distinta: algunos priorizan la raíz, otros los subárboles, y otros procesan la información por niveles.

El presente trabajo se enfoca en implementar estos recorridos en C++ y Java. La implementación permite reforzar la comprensión teórica mediante la práctica, observando cómo cambia el orden de salida cuando se agregan nuevos nodos al árbol. Además, se incluyen funciones adicionales para contar nodos y hojas, lo cual ayuda a analizar mejor la estructura construida.

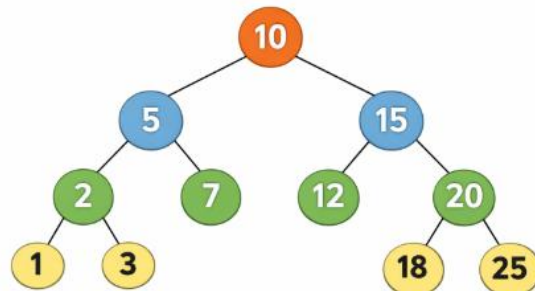
También se aplica el concepto de árbol binario a un sistema de registro de asistencia. En este caso, los módulos del sistema pueden verse como nodos relacionados jerárquicamente. Por ejemplo, un sistema puede tener módulos principales como usuarios y asistencias, y cada uno puede dividirse en submódulos como registrar, buscar, marcar asistencia o generar reportes.

4. Metodología y Diseño

Para el desarrollo de la práctica se construyó inicialmente un árbol binario con la raíz 10. El nodo 10 tiene como hijos a 5 y 15. A su vez, el nodo 5 tiene como hijos a 2 y 7, mientras que el nodo 15 tiene como hijos a 12 y 20. Este árbol permitió aplicar los recorridos básicos y observar el orden en el que cada algoritmo visita los nodos.



Posteriormente, se modificó el árbol agregando los nodos 1, 3, 18 y 25. Los nodos 1 y 3 fueron ubicados como hijos del nodo 2, mientras que los nodos 18 y 25 fueron ubicados como hijos del nodo 20. Con esta modificación, el árbol aumentó su cantidad de nodos y permitió comprobar nuevamente los recorridos.



El árbol modificado quedó conformado por 11 nodos. Los recorridos obtenidos fueron: preorden, inorden, postorden y BFS. Además, mediante el método de conteo de hojas se determinó que el árbol posee 6 hojas, correspondientes a los nodos 1, 3, 7, 12, 18 y 25.

5. Implementación del código

En C++ se utilizó una estructura llamada Nodo, la cual contiene un dato entero y dos punteros: uno para el hijo izquierdo y otro para el hijo derecho. Esta forma de implementación permite enlazar manualmente cada nodo del árbol mediante direcciones de memoria. El uso de punteros es

una característica importante de C++, ya que permite controlar de forma directa la relación entre los nodos.

```
// Estructura de nodo para valores enteros
struct Nodo {
    int dato;
    Nodo* izquierda;
    Nodo* derecha;

    Nodo(int valor) {
        dato = valor;
        izquierda = nullptr;
        derecha = nullptr;
    }
};
```

En Java se utilizó una clase llamada `Nodo`, que contiene un atributo entero y dos referencias hacia otros objetos de tipo `Nodo`. A diferencia de C++, Java no utiliza punteros explícitos, sino referencias a objetos. Esto hace que la implementación sea más sencilla de leer para algunos programadores, ya que la administración de memoria se maneja automáticamente mediante el recolector de basura.

```
// Clase nodo para valores enteros
class Nodo {
    int dato;
    Nodo izquierda;
    Nodo derecha;

    public Nodo(int dato) {
        this.dato = dato;
        this.izquierda = null;
        this.derecha = null;
    }
}
```

El método `preorden` visita primero la raíz del árbol, luego recorre el subárbol izquierdo y finalmente el subárbol derecho. En el árbol modificado, este método muestra primero el valor 10, luego continúa con los nodos del lado izquierdo y después con los del lado derecho. Este recorrido es útil cuando se desea procesar primero el elemento principal antes que sus dependientes.

```
// Recorrido preorden: primero raíz
void preorden(Nodo* raiz) {
    if (raiz == nullptr) return;
    cout << raiz->dato << " ";
    preorden(raiz->izquierda);
    preorden(raiz->derecha);
}
```

El método `inorden` primero recorre el subárbol izquierdo, luego visita la raíz y finalmente recorre el subárbol derecho. En un árbol binario de búsqueda, este recorrido permite obtener los valores ordenados de menor a mayor. En la práctica realizada, el resultado del recorrido `inorden` muestra

los valores en orden ascendente porque el árbol conserva una organización compatible con un árbol de búsqueda.

```
// Recorrido inorden: raíz en medio
void inorden(Nodo* raiz) {
    if (raiz == nullptr) return;
    inorden(raiz->izquierda);
    cout << raiz->dato << " ";
    inorden(raiz->derecha);
}
```

El método *postorden* recorre primero el subárbol izquierdo, luego el subárbol derecho y al final visita la raíz. Este recorrido es útil cuando se necesita procesar primero los nodos dependientes antes que el nodo principal. Por ejemplo, puede aplicarse en tareas donde se deben cerrar o eliminar procesos hijos antes de finalizar el proceso principal.

```
// Recorrido postorden: raíz al final
void postorden(Nodo* raiz) {
    if (raiz == nullptr) return;
    postorden(raiz->izquierda);
    postorden(raiz->derecha);
    cout << raiz->dato << " ";
}
```

El método *bfs*, también llamado recorrido por niveles que utiliza una cola para visitar los nodos desde la raíz hacia los niveles inferiores. Primero procesa el nodo raíz, luego sus hijos, después los hijos de esos hijos, y así sucesivamente. Este recorrido es adecuado para mostrar información jerárquica nivel por nivel, como menús principales y submenús dentro de un sistema.

```
// Recorrido por niveles usando cola (BFS)
void bfs(Nodo* raiz) {
    if (raiz == nullptr) return;

    queue<Nodo*> cola;
    cola.push(raiz);

    while (!cola.empty()) {
        Nodo* actual = cola.front();
        cola.pop();

        cout << actual->dato << " ";

        if (actual->izquierda != nullptr) cola.push(actual->izquierda);
        if (actual->derecha != nullptr) cola.push(actual->derecha);
    }
}
```

El método *contarNodos* calcula la cantidad total de nodos del árbol. Funciona de manera recursiva: si el nodo está vacío, retorna 0; si existe, cuenta el nodo actual y suma los nodos del subárbol izquierdo y derecho. En el árbol modificado, este método devuelve 11, que corresponde a todos los nodos existentes.

```
// Cuenta todos los nodos del árbol
int contarNodos(Nodo* raiz) {
    if (raiz == nullptr) return 0;
    return 1 + contarNodos(raiz->izquierda) + contarNodos(raiz->derecha);
}
```

El método *contarHojas* permite contar los nodos que no tienen hijos. Primero verifica si el nodo está vacío; luego comprueba si no tiene hijo izquierdo ni derecho. Si cumple esa condición, se considera una hoja. En la práctica, este método devuelve 6, ya que los nodos hoja son 1, 3, 7, 12, 18 y 25.

```
// Cuenta solo los nodos hoja
int contarHojas(Nodo* raiz) {
    if (raiz == nullptr) return 0;
    if (raiz->izquierda == nullptr && raiz->derecha == nullptr) return 1;
    return contarHojas(raiz->izquierda) + contarHojas(raiz->derecha);
}
```

Para el ejercicio aplicado al proyecto final, se representó un sistema de registro de asistencia como un árbol binario. La raíz puede ser el módulo principal del sistema, mientras que los hijos representan módulos como usuarios y asistencias. A su vez, estos módulos pueden dividirse en registrar, buscar, marcar asistencia y reportes. Esta representación permite comprender cómo los árboles ayudan a organizar sistemas de manera jerárquica.

6. Resultados

Tabla 1. Recorridos del árbol original.

Recorrido	Resultado
Preorden	10 5 2 7 15 12 20
Inorden	2 5 7 10 12 15 20
Postorden	2 7 5 12 20 15 10
BFS	10 5 15 2 7 12 20

Tabla 2. Recorridos del árbol modificado.

Recorrido	Resultado
Preorden	10 5 2 1 3 7 15 12 20 18 25
Inorden	1 2 3 5 7 10 12 15 18 20 25
Postorden	1 3 2 7 5 12 18 25 20 15 10

BFS	10 5 15 2 7 12 20 1 3 18 25
-----	-----------------------------

En el árbol modificado se obtuvo un total de 11 nodos y 6 hojas. Las hojas identificadas fueron 1, 3, 7, 12, 18 y 25.

RECORRIDOS DE ARBOLES BINARIOS – UTA

EJERCICIO 1: Arbol original

Preorden: 10 5 2 7 15 12 20

Inorden: 2 5 7 10 12 15 20

Postorden: 2 7 5 12 20 15 10

BFS: 10 5 15 2 7 12 20

EJERCICIO 2: Arbol modificado

Preorden: 10 5 2 1 3 7 15 12 20 18 25

Inorden: 1 2 3 5 7 10 12 15 18 20 25

Postorden: 1 3 2 7 5 12 18 25 20 15 10

BFS: 10 5 15 2 7 12 20 1 3 18 25

EJERCICIO 3: Total de nodos

Total de nodos: 11

EJERCICIO 4: Total de hojas

Total de hojas: 6

EJERCICIO 5: Sistema web como arbol binario

Preorden: Sistema_Web Usuarios Registrar Buscar Inventario Productos Reportes

Postorden: Registrar Buscar Usuarios Productos Reportes Inventario Sistema_Web

BFS: Sistema Web Usuarios Inventario Registrar Buscar Productos Reportes

7. Diferencias entre C++ y Java

Una diferencia importante entre C++ y Java es la forma en que manejan la memoria. En C++, los nodos se crean con `new` y se accede a ellos mediante punteros. Esto permite mayor control, pero también exige más cuidado para evitar errores de memoria. En Java, los nodos también se crean con `new`, pero se manejan mediante referencias y la memoria es administrada automáticamente.

Otra diferencia se encuentra en la sintaxis. En C++ se utiliza `Nodo*` para indicar que una variable es un puntero a un nodo. En Java no se usa el símbolo de puntero, sino que se declara directamente el tipo de objeto. Por ejemplo, en Java se escribe `Nodo izquierda`, mientras que en C++ se escribe `Nodo* izquierda`.

También existe diferencia en las bibliotecas utilizadas para BFS. En C++ se usó la librería `queue`, mientras que en Java se utilizó `Queue` junto con `LinkedList`. Aunque ambos lenguajes permiten implementar una cola, Java requiere importar clases del paquete `java.util`, mientras que C++ utiliza la biblioteca estándar con `#include <queue>`.

En cuanto a la ejecución, C++ compila el programa en un archivo ejecutable, mientras que Java compila el código a bytecode y luego lo ejecuta mediante la máquina virtual de Java. Esto hace

que Java sea más portable entre sistemas, mientras que C++ suele ser más cercano al sistema operativo donde se compila.

A pesar de estas diferencias, la lógica de los recorridos es prácticamente la misma en ambos lenguajes. Los métodos recursivos mantienen la misma estructura: verificar si el nodo es nulo, procesar el nodo actual y llamar recursivamente a los hijos izquierdo y derecho según el tipo de recorrido.

8. Conclusiones

La implementación de los recorridos de árboles binarios permitió comprender el funcionamiento de estructuras jerárquicas en programación. Al aplicar preorden, inorden, postorden y BFS se evidenció que cada algoritmo visita los nodos en un orden diferente. Esto demuestra que los recorridos no son intercambiables, sino que deben seleccionarse según la necesidad del problema.

El árbol modificado permitió comprobar el funcionamiento de las funciones adicionales de conteo. La función contarNodos determinó que el árbol tenía 11 nodos en total, mientras que la función contarHojas identificó 6 hojas. Estos resultados confirman que la recursividad es una herramienta adecuada para analizar estructuras de árbol.

Al comparar C++ y Java se observó que ambos lenguajes permiten resolver el mismo problema, pero con diferencias en sintaxis, manejo de memoria y uso de bibliotecas. C++ requiere trabajar con punteros, mientras que Java utiliza referencias y administra la memoria automáticamente. Sin embargo, la lógica de los métodos es equivalente en ambos casos.

La aplicación del árbol binario a un sistema de registro de asistencia permitió relacionar la teoría con un caso práctico. Los módulos de un sistema pueden organizarse jerárquicamente, y los recorridos permiten decidir cómo mostrar, procesar o analizar esa estructura. Por ejemplo, BFS es útil para mostrar módulos por niveles, mientras que postorden ayuda a procesar primero elementos internos.

El sistema puede representarse con una raíz llamada Sistema_Asiencia. Sus hijos principales pueden ser Usuarios y Asistencias. A su vez, Usuarios puede contener Registrar y Buscar, mientras que Asistencias puede contener Marcar y Reportes.

Para mostrar el menú principal se recomienda preorden, porque presenta primero el módulo raíz y después sus submódulos. Para procesar primero módulos internos se recomienda postorden, porque atiende los nodos hijos antes del módulo principal. Para mostrar módulos nivel por nivel se recomienda BFS, debido a que recorre desde la raíz hacia los niveles inferiores.

El recorrido preorden es útil cuando se necesita mostrar primero la estructura general del sistema. Inorden resulta especialmente útil en árboles binarios de búsqueda, ya que puede devolver los valores en orden ascendente. Postorden permite procesar primero los nodos dependientes antes del nodo principal. Finalmente, BFS es apropiado para visualizar niveles jerárquicos.

Finalmente, la práctica fortaleció la comprensión de estructuras de datos y permitió observar cómo una misma solución puede implementarse en dos lenguajes diferentes. Esto es importante en la formación en software, ya que ayuda a desarrollar pensamiento lógico independiente del lenguaje utilizado.

9. Recomendaciones

Se recomienda practicar los recorridos con árboles de diferentes tamaños para comprender mejor cómo cambia el orden de salida. Mientras más nodos tenga el árbol, más evidente se vuelve la diferencia entre preorden, inorden, postorden y BFS.

También se recomienda dibujar el árbol antes de programarlo. Esto ayuda a identificar correctamente la raíz, los hijos izquierdos, los hijos derechos y las hojas. De esta manera se evitan errores al enlazar los nodos dentro del código.

En C++ se recomienda tener cuidado con el uso de punteros, ya que un puntero mal asignado puede generar errores durante la ejecución. También sería conveniente liberar la memoria utilizada si el programa crece o se convierte en una aplicación más grande.

En Java se recomienda aprovechar las clases disponibles en `java.util`, como `Queue` y `LinkedList`, porque facilitan la implementación de recorridos por niveles. Además, se recomienda mantener nombres claros en los métodos para que el código sea fácil de leer.

Para futuras mejoras, se recomienda implementar inserción automática de nodos, búsqueda de valores, eliminación de nodos y visualización gráfica del árbol. Estas mejoras permitirían convertir la práctica en una aplicación más completa para el estudio de estructuras de datos.

10. Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014.

[3] Oracle, “The Java Tutorials: Collections Framework,” Oracle, 2026. [Online]. Available: <https://docs.oracle.com/javase/tutorial/collections/>

[4] `cppreference.com`, “`std::queue`,” 2026. [Online]. Available: <https://en.cppreference.com/w/cpp/container/queue>

[5] R. Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis, IN, USA: Sams Publishing, 2002.

[6] J. A. Redroban Chimbo, “Recorridos-de-arboles-UTA,” GitHub repository, 2026. [Online]. Available: <https://github.com/JoselynREDRO106/Recorridos-de-arboles-UTA>